A Theory and Implementation of General Intelligence: From Cognitive Architecture to Embodied Agents

Zhongyin Cai

Abstract

The fundamental inquiry into the nature of intelligence and its artificial realization lies at the heart of AI research. From the perspective of cognitive science, this paper proposes the core thesis that "Intellect is mapping" and systematically elaborates a theoretical framework in which intellect is understood as the capacity for information filtering, compression, storage, association, transformation, and generation.

$$\mathcal{I} : \mathcal{X} \to \mathcal{Y}, \qquad \mathcal{Y} = \mathcal{I}(\mathcal{X}), \tag{1}$$

where $\mathcal{X}$ denotes the input information space and $\mathcal{Y}$ the output information space. We decompose intelligence into the synergistic interplay of intellect, knowledge, and resources.

$$\mathcal{C} = \mathcal{F}\left(\mathcal{I}, K, R\right), \tag{2}$$

where K is knowledge and R denotes resources. On this basis, we construct a modular architecture for general-purpose agent, encompassing elemental intellectual faculties such as attention, perception, memory, search, computation, and imagination, as well as their combinations—recognition and creativity. We further discuss implementation pathways across three levels: embodied intelligence, collective intelligence, and machine intelligence. Special attention is given to the triadic technology stack of data–algorithm–compute, analyzing the full closed loop from data acquisition, cleaning, mining, and annotation to simulation training and real-robot deployment, along with the engineering realization of core modules including perception, planning, control, end-to-end learning, and language understanding and generation. Through a comparative analysis of symbolism and connectionism, we argue for the viability of neuro-symbolic fusion architectures as a pathway to general intelligence, and establish a five-dimensional evaluation system for intelligence in terms of precision, speed, depth, breadth, and efficiency.

$$\mathbf{Q} = \left(P, S, D, B, E\right), \qquad P, S, D, B, E \in \mathbb{R}_{+}, \tag{3}$$

where the five components are precision, speed, depth, breadth, and efficiency, respectively. Finally, we point out that building a general-purpose brain with full perception, planning, control, autonomous exploration, and continual learning capabilities, combined with diverse effectors and rapid iteration in real-world scenarios, constitutes a pragmatic path toward artificial general intelligence.

Keywords: intelligence theory; cognitive architecture; embodied AI; neuro-symbolic AI; general-purpose agent; continual learning

1 Introduction

Since the birth of artificial intelligence, the questions of "what is intelligence" and "how to realize it" have never ceased. From the Turing test to symbolism, from connectionism to today's neuro-symbolic fusion, every theoretical breakthrough and technological advance has redefined the boundaries of intelligence. However, despite deep learning and large language models demonstrating superhuman performance on specific tasks, the realization of general intelligence still faces fundamental challenges.

A notable characteristic of current AI research is that basic modules are becoming mature, yet system-level integration remains insufficient. Perception models have made great strides in vision, text, speech, and other modalities; large language models exhibit surprising reasoning and planning abilities; and reinforcement learning has reached superhuman levels in certain control tasks [13]. Nevertheless, organically integrating these modules into a single general-purpose agent that can continually learn and autonomously make decisions in open, unstructured environments remains an open problem [3][4][5][10].

The goal of this paper is to construct a complete theoretical framework from the fundamental definition of intelligence to its engineering implementation. Our core view is that the essence of intelligence is mapping—the transformation of information into features, and features into other information or new information.

$$\mathbf{x} \xrightarrow{\text{feature extraction}} \mathbf{f}, \qquad \mathbf{f} \xrightarrow{\text{information processing}} \mathbf{x}', \tag{4}$$

where $\mathbf{x}$ is raw information, $\mathbf{f}$ is a feature representation, and $\mathbf{x}'$ denotes other information or newly generated information. Intellect, as meta-cognitive capacity, underlies all concrete abilities; capabilities, in turn, are the synergistic product of intellect, knowledge, and resources.

$$C = \mathcal{F}(\mathcal{I}, K, R), \tag{5}$$

where $\mathcal{I}$ denotes intellect, K knowledge, and R resources. Based on this definition, we can decompose a general-purpose agent into engineering-realizable functional modules and systematically examine the achievements and shortcomings of existing technologies along the data–algorithm–compute axis.

The contributions of this paper are:

1. Proposing "intellect as mapping" as a unified theoretical framework, incorporating elemental faculties—attention, perception, memory, search, computation, and imagination—into a coherent cognitive architecture.

2. Constructing a complete mapping from cognitive modules to engineering implementations, providing a reference architecture for the design of general-purpose agents.

3. Systematically analyzing key technical challenges and solution paths at the three levels of data engineering, algorithm design, and compute deployment.

4. Establishing a multi-dimensional evaluation system for intelligence and offering a comparative analysis between human and machine intelligence.

2 Theoretical Foundations of Intelligence

2.1 Definition of Intelligence: Intellect and Capability

Intelligence is a multi-layered concept. From a functionalist perspective, it can be decomposed into two dimensions: intellect and capability. Intellect is the core of intelligence—the meta-cognitive capacity for processing information; capability is the external manifestation of intellect when combined with knowledge and resources in specific contexts.

$$\text{Intelligence} = (\text{intellect},\ \text{capability}), \qquad C = \mathcal{F}(\mathcal{I}, K, R), \tag{6}$$

where the first component is the meta-cognitive capacity for information processing and the second is its external manifestation given knowledge K and resources R.

2.2 Intellect as Mapping

We propose the core thesis that *Intellect is mapping*: the ability to transform information from one form to another.

$$\mathcal{I} : \mathcal{X} \to \mathcal{Y}, \qquad \mathbf{y} = \mathcal{I}(\mathbf{x}), \tag{7}$$

where $\mathbf{x} \in \mathcal{X}$ is information in one form and $\mathbf{y} \in \mathcal{Y}$ is information in another form. Specifically, Intellect comprises the following basic operations:

1. **Attention**: information weighting, filtering out irrelevant information;

$$\mathbf{x}_{\text{att}} = \mathcal{A}(\mathbf{x}, \mathbf{q}; \mathbf{K}, \mathbf{V}) = \sum_i \alpha_i \mathbf{v}_i, \qquad \alpha_i = \text{softmax}\left(\frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d_k}}\right), \tag{8}$$

   where $\mathbf{q}$, $\mathbf{k}_i$, and $\mathbf{v}_i$ are query, key, and value vectors.

2. **Perception**: extracting features from raw information;

$$\mathbf{f} = \mathcal{P}(\mathbf{x}; \theta_p), \qquad \mathbf{f} \in \mathbb{R}^d, \tag{9}$$

   where θ_p denotes the parameters of the perception model.

3. **Memory**: storing features in retrievable forms;

$$\mathcal{M} :\ \mathbf{f} \ \mapsto\ (\text{key}, \mathbf{f}), \qquad \mathbf{f} \in \text{RetrievableStore}(\mathcal{M}), \tag{10}$$

   where the key enables future retrieval of the stored feature.

4. **Search**: establishing associations among features and retrieving them;

$$\mathbf{f}_{\text{ret}} = \mathcal{S}(\mathbf{f}_q) \ =\ \arg\max_{\mathbf{f}_i \in \mathcal{M}} \text{sim}(\mathbf{f}_q, \mathbf{f}_i), \tag{11}$$

   where $\text{sim}(\cdot, \cdot)$ is a similarity measure (e.g., cosine similarity).

5. **Computation**: transforming perceptual features, associative features, and operational symbols into new patterns;

$$\mathbf{y} = \mathcal{C}(\mathbf{f}_1, \mathbf{f}_2, \ldots, \mathbf{f}_n, \mathbf{s}; \theta_c), \tag{12}$$

   where $\mathbf{f}_1, \ldots, \mathbf{f}_n$ are perceptual or associative features, $\mathbf{s}$ denotes operational symbols, and θ_c represents computational parameters.

6. **Imagination**: recombination, fusion, and generation of features;

$$\mathbf{f}_{\text{new}} = \mathcal{G}(\mathbf{f}_1, \mathbf{f}_2, \ldots; \theta_g), \tag{13}$$

where θ_g denotes the parameters of the generative model.

Thus, the set of basic intellectual operations is

$$\mathcal{I} = \{\mathcal{A}, \mathcal{P}, \mathcal{M}, \mathcal{S}, \mathcal{C}, \mathcal{G}\}. \tag{14}$$

These basic intellectual operations can be combined into higher-level cognitive functions. *Recognition* is the combination of attention, perception, memory, and search—from information to features to memory association.

$$\mathbf{x} \xrightarrow{\mathcal{A}} \mathbf{x}_{\text{att}} \xrightarrow{\mathcal{P}} \mathbf{f} \xrightarrow{\mathcal{M}} \text{store}(\mathbf{f}) \xrightarrow{\mathcal{S}} \mathbf{f}_{\text{ret}} \rightarrow \text{label/entity}. \tag{15}$$

Creativity further incorporates computation, imagination, and validation—including both discovery-based innovation (discovering new features) and combinatorial innovation (generating new combinations), both requiring validation with real data.

$$\mathbf{f}_{\text{new}} = \mathcal{G}\left(\mathcal{C}\left(\mathbf{f}_{\text{att}}, \mathbf{f}_{\text{ret}}, \ldots\right)\right), \qquad \mathcal{V}(\mathbf{f}_{\text{new}}, \mathcal{D}_{\text{real}}) \in \{0, 1\}, \tag{16}$$

where $\mathcal{V}$ is validation against real data $\mathcal{D}_{\text{real}}$ and a value of 1 denotes acceptance.

2.3 Capability: Synergy of Intellect, Knowledge, and Resources

If intellect is the "hardware" foundation of intelligence, then capability is the "software" manifestation when intellect is combined with knowledge and resources.

$$C = \mathcal{F}\left(\mathcal{I}, K, R\right), \tag{17}$$

Knowledge can be viewed as the integral of intellect over time—accumulation of intellect; intellect is the gradient of knowledge—the direction and rate of knowledge change.

$$K(t) = K(t_0) + \int_{t_0}^{t} \mathcal{I}(\tau)\, \mathrm{d}\tau, \tag{18}$$

$$\mathcal{I}(t) = \frac{\mathrm{d}K}{\mathrm{d}t}. \tag{19}$$

The concrete expression of capability requires the support of resources, including matter, energy, information, time, and space. In the context of embodied intelligence [6][7], basic capabilities include:

- Proprioception: perception of one's own state (body, actions, emotions);
- Environmental perception: spatial localization and recognition of objects and actions;
- Motion capabilities: planning, execution, evaluation, and correction;
- Language capabilities: listening, speaking, reading, writing;

- Collective intelligence: state communication, multi-robot scheduling, human-robot collaboration.

These basic capabilities further combine to form advanced domain-specific intelligence —professional intelligence in sports, mathematics, science, technology, economics, art, and other fields.

$$\mathcal{D}_j = \mathcal{H}\left(C_{\text{prop}}, C_{\text{env}}, C_{\text{mot}}, C_{\text{lang}}, C_{\text{col}}\right), \tag{20}$$

where $\mathcal{D}_j$ denotes the intelligence of a professional domain and $\mathcal{H}$ is the combination function over the basic embodied capabilities.

2.4 Bridging Symbolism and Connectionism

In the history of AI, symbolism and connectionism have constituted two major paradigmatic paths [9]. Symbolism holds that the world can be explained through symbols and processed through precise logical procedures; connectionism argues that this process should be realized via artificial neural networks. Each has its advantages: symbols are naturally efficient for logical computation, while neural networks excel in pattern recognition and analog computation.

$$\text{Symbolism}: \ \mathbf{y} = \text{Logic}\big(\text{Symbolize}(\mathbf{x})\big), \qquad \text{Connectionism}: \ \mathbf{y} = \text{NN}_\theta(\mathbf{x}). \tag{21}$$

However, pure symbolism struggles to learn from data and handle ambiguity and uncertainty; pure neural networks are "black boxes," difficult to interpret and heavily data-dependent. Neuro-symbolic AI emerges precisely to combine the strengths of both—neural networks for pattern recognition and learning, symbolic reasoning for structured decision-making and interpretability [8][15].

Our key insight is that all symbols can be tokenized and then processed through neural networks via analog computation. This means symbols and neural networks are not opposed but can coexist within a unified architecture—symbols providing efficient knowledge representation and logical reasoning, neural networks providing flexible pattern matching and generation.

$$\text{Neuro-symbolic}: \quad \mathbf{s} \xrightarrow{\text{Tokenize}} \mathbf{t}(\mathbf{s}) \xrightarrow{\text{Embed}} \mathbf{e}_\mathbf{s} \xrightarrow{\text{NN}} \mathbf{h} \xrightarrow{\text{SymbolicReasoning}} \mathbf{y}. \tag{22}$$

This fusion architecture provides the theoretical foundation for designing general-purpose agents.

3 Implementation of Intelligence: Architecture, Data, Algorithms, and Compute Engineering

3.1 Modular Architecture of General-Purpose Agents

The design of general-purpose agents can follow modular principles. Contemporary robot frameworks typically comprise perception, planning, and control [12]. Based on the intellectual faculties proposed earlier, we can map these components to specific intellectual functions.

- **Attention Module**: receives information and queries, outputs relevant information. In engineering, this can be realized via neural attention mechanisms.

$$\mathbf{q} = \text{Query}(\mathbf{x}), \quad \mathbf{k}_i = \text{Key}(\mathbf{m}_i), \quad \mathbf{v}_i = \text{Value}(\mathbf{m}_i), \tag{23}$$

$$\mathbf{o} = \sum_i \alpha_i \mathbf{v}_i, \qquad \alpha_i = \text{softmax}\big(\mathbf{q}^\top \mathbf{k}_i\big), \tag{24}$$

  where $\{\mathbf{m}_i\}$ denotes memory content or source information.

- **Perception Module**: converts raw information into numeric vectors/matrices via a tokenizer, then extracts features via an encoder.

$$\mathbf{f} = \text{Encoder}\big(\text{Tokenizer}(\mathbf{x})\big). \tag{25}$$

- **Memory Module**: requires coordination of multiple storage structures: vector databases for raw information and associative features (supporting similarity retrieval); relational databases for structured relationships between information and symbols; graph databases for association structures among information.

$$\mathcal{M}_{\text{vec}}: \ \mathbf{f} \mapsto (\text{id}, \mathbf{f}), \quad \mathcal{M}_{\text{rel}}: \ (e_1, e_2, r), \quad \mathcal{M}_{\text{graph}}: \ \mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{R}), \tag{26}$$

  where $\mathcal{V}$, $\mathcal{E}$, and $\mathcal{R}$ denote information nodes, associations, and relation types, respectively.

- **Search Module**: computes similarity between perceptual feature matrices and memory feature matrices to retrieve and associate features.

$$\mathbf{S} = \mathbf{F}_{\text{percept}}\, \mathbf{F}_{\text{memory}}^\top, \qquad \mathbf{f}_{\text{ret}} = \arg\max_{\mathbf{f}_i \in \mathcal{M}} \ \text{sim}(\mathbf{f}_q, \mathbf{f}_i), \tag{27}$$

  The importance of searching for information is determined jointly by the importance of the verified data, the similarity of the information before, and the referencing relationship.

$$w_i = \psi\big(\text{imp}(d_i), \ \text{sim}(q, d_i), \ \text{ref}(d_i)\big), \tag{28}$$

  where $\text{imp}(d_i)$ is the importance of verified data, $\text{sim}(q, d_i)$ is the similarity between the current information and retrieved information, and $\text{ref}(d_i)$ measures the referencing relationship.

- **Computation Module**: transforms entity features, associative features, and operational symbols into output patterns.

$$\mathbf{y} = \mathcal{C}\left(\mathbf{f}_{\text{entity}}, \mathbf{f}_{\text{assoc}}, \mathbf{s}_{\text{op}}\right). \tag{29}$$

- **Imagination Module**: fuses, recombines, and generates new feature matrices from entity and associative features, then combines with memory to produce novel information.

$$\mathbf{f}_{\text{new}} = \mathcal{G}\left(\mathbf{f}_{\text{entity}}, \mathbf{f}_{\text{assoc}}\right), \qquad \mathbf{x}_{\text{novel}} = \text{Decode}\left(\mathbf{f}_{\text{new}}, \mathcal{M}\right). \tag{30}$$

Combinations of these modules yield higher-level intelligence:

- *Recognition* is the complete chain information → features → memory → association, converting images/point-clouds/sounds into recognized entities/language/numbers.

$$\begin{aligned}\text{Recognition} : \mathbf{x} \;\to\; \mathbf{f} \;\to\; \mathcal{M}(\mathbf{f}) \;\to\; \mathbf{f}_{\text{assoc}} \\ \to \;\text{entities/language/numbers},\end{aligned} \tag{31}$$

- *Innovation* includes combinatorial and discovery-based forms:
  - Combinatorial: information → attention → features → memory → association → transformation → new information generation → real-data validation → effective generation.

$$\mathbf{x} \to \mathcal{A}(\mathbf{x}) \to \mathcal{P}(\mathbf{x}) \to \mathcal{M} \to \mathcal{S} \to \mathcal{C} \to \mathcal{G} \xrightarrow{\mathcal{V}(\cdot,\mathcal{D}_{\text{real}})} \text{effective generation}. \tag{32}$$

  - Discovery-based: autonomous exploration → attention → new information → features → memory → association → new information discovery → real-data validation → effective discovery.

$$\text{exploration} \to \mathcal{A} \to \mathbf{x}_{\text{new}} \to \mathcal{P} \to \mathcal{M} \to \mathcal{S} \xrightarrow{\mathcal{V}(\cdot,\mathcal{D}_{\text{real}})} \text{effective discovery}. \tag{33}$$

3.2 Data Engineering

In real-world robotics projects, algorithmic models are often not the primary bottleneck—rather, the key to success lies in whether sufficiently rich data can be obtained, whether valuable mapping relationships exist within the data, and the efficiency of data iteration.

$$\mathcal{T}_{\text{system}} = \mathcal{T}\big(D_{\text{richness}},\ \text{map}(D),\ \eta_{\text{iteration}}\big), \tag{34}$$

where D_{richness} measures data abundance, $\text{map}(D)$ measures the value of the mapping relationships contained in the data, and $\eta_{\text{iteration}}$ measures data-iteration efficiency.

3.2.1 Data Acquisition

Collection: images, text, sound, tactile, etc. Data augmentation: random scaling, cropping, rotation, gamma transformation, affine/perspective transforms, mosaic, etc. Scanning + rendering (BlenderProc). Generation (GAN, diffusion) [11]. Reconstruction (SAM3D, VGGT, DA3, NeRF, 3DGS). Simulation (PyBullet, Gazebo, MuJoCo, Isaac-Sim).

3.2.2 Data Cleaning

Data quality assessment; removal of dirty data (weak or missing mapping between input and label); deduplication of similar features; format conversion for various model training and inference pipelines.

$$D_{\text{clean}} = \big\{(x_i, y_i) \in D :\ q(x_i, y_i) \geq \tau_q\big\} \ \setminus\ \big\{(x_i, y_i) : \exists\, j \neq i,\ \text{sim}(\mathbf{f}_i, \mathbf{f}_j) > \tau_s\big\}, \tag{35}$$

where $q(x_i, y_i)$ denotes the quality of the input–label mapping and τ_q, τ_s are thresholds.

3.2.3 Data Mining

Feature matrix $\times$ matrix$^\top$ $\rightarrow$ similarity matrix $\rightarrow$ for labeled data: low similarity within same class (outliers) and high similarity across different classes (confusing points); for unlabeled data: clustering connected components of similar features and generating outliers; also, low similarity among outputs from different model structures for the same class (disputed points).

$$\mathbf{S} = \mathbf{F}\mathbf{F}^\top, \qquad \mathbf{F} \in \mathbb{R}^{n\times d}, \quad \mathbf{S} \in \mathbb{R}^{n\times n}, \tag{36}$$

$$\begin{aligned} \mathcal{O}_{\text{labeled}} &= \big\{(i,j) :\ y_i = y_j,\ S_{ij} < \tau_{\text{low}}\big\}, \\ \mathcal{P}_{\text{confusing}} &= \big\{(i,j) :\ y_i \neq y_j,\ S_{ij} > \tau_{\text{high}}\big\}, \\ \mathcal{O}_{\text{unlabeled}} &= \text{outliers of connected-component clustering on } \mathbf{S}. \end{aligned} \tag{37}$$

3.2.4 Data Annotation

Hybrid approach of model pre-annotation and manual annotation, managed with tools like Label-Studio and CVAT.

$$A_{\text{final}} = A_{\text{model}} \cup A_{\text{manual}}, \qquad A_{\text{manual}} \supseteq \big\{(x,y) \in A_{\text{model}} :\ \text{conf}_{\text{model}}(x) < \tau\big\}. \tag{38}$$

3.2.5 Data Verification

Algorithmic verification: sampling low-confidence predictions, disputed points between algorithm and manual labels; Manual verification: checking agreement with preset samples (known correct labels mixed with a few incorrect ones), sampling multi-annotator disagreements, random sampling.

$$\mathcal{V}_{\text{alg}} = \big\{(x,y) :\ p_\theta(y \mid x) < \tau_{\text{conf}}\big\} \cup \big\{\text{algorithm–manual disputes}\big\}, \tag{39}$$

$$\mathcal{V}_{\text{manual}} = \big\{(x,y) :\ \text{disagreement with preset checks, annotators, or random samples}\big\}. \tag{40}$$

3.2.6 Generalization

Closed-set fixed scenes: variations in angle, position, illumination, occlusion, blur. Closed-set multi-scenes: switching scenes for already trained categories. Open-set multi-scenes: registering unseen categories/scenes via feature extraction and matching for similarity-based generalization.

$$\hat{y} = \arg\max_{c\in\mathcal{C}_{\text{known}}} \ \text{sim}\big(\mathbf{f}(x), \boldsymbol{\mu}_c\big), \qquad \begin{cases} \text{recognize } \hat{y}, & \text{sim} \geq \tau_{\text{sim}}, \\ \text{register a new category/scene}, & \text{sim} < \tau_{\text{sim}}, \end{cases} \tag{41}$$

where $\boldsymbol{\mu}_c$ is the prototype or registration-set feature of category c.

3.2.7 Real-Robot Closed-Loop Testing and Iteration

Missed/false detections: known environment + dynamic reasoning—model dynamically stores missed detection samples; for false detections, stores false positives based on known object categories.

$$D_{\text{closed-loop}} \leftarrow D_{\text{closed-loop}} \cup D_{\text{missed}} \cup D_{\text{false-positive}}, \tag{42}$$

where D_{missed} contains dynamically stored missed-detection samples and $D_{\text{false-positive}}$ stores false positives according to known categories. Localization bias: tactile feedback + dynamic reasoning—model-predicted trajectory executed with tactile-based calibration (over-correction/under-correction compensation).

$$\hat{\mathbf{p}} = \mathbf{p}_{\text{model}} + \boldsymbol{\delta}_{\text{tactile}}, \qquad \boldsymbol{\delta}_{\text{tactile}} = \varphi\big(\text{over-/under-correction compensation}\big). \tag{43}$$

3.2.8 Real-to-Sim-to-Real Pipeline:

1. Batch collect human-ego operation data (multi-view video, task language descriptions, operator pose, arm end-effector pose and hand joint-angle trajectories, tactile sensor information).

2. Collect a small amount of real-robot teleoperation data (multi-view video, task descriptions, robot pose, arm end-effector pose, dexterous hand joint-angle trajectories, tactile info).

3. Use VLM model to decompose human and robot action videos into segments of primitive actions, with manual verification of decomposition quality.

4. Train robot action model taking multi-view video, primitive action type, robot pose, end-effector pose, hand joint trajectories, and tactile info as input; use multiple encoders to extract multimodal features, fuse them, and feed into an action expert head to output future trajectories, a value network to predict success/failure probability, and a feature prediction head to predict future world state changes.

$$\begin{aligned}\big(\hat{\boldsymbol{\tau}}_{t:t+H},\ \hat{v},\ \hat{\mathbf{z}}_{t:t+H}\big) = \mathcal{E}_{\text{action}}\Big(&\mathcal{E}_{\text{fusion}}\big(\mathcal{E}_{\text{mv}}(I_{\text{mv}}), \mathcal{E}_{\text{act}}(a_{\text{prim}}),\\ &\mathcal{E}_{\text{pose}}(\mathbf{p}), \mathcal{E}_{\text{hand}}(\mathbf{q}_h), \mathcal{E}_{\text{tac}}(\mathbf{s}_{\text{tac}})\big)\Big),\end{aligned} \tag{44}$$

where $\hat{\boldsymbol{\tau}}$ is the future trajectory head, $\hat{v}$ is the success/failure value head, and $\hat{\mathbf{z}}$ is the future world-state feature head.

5. Modeling and simulation:

- **5.1** Generate 3D models of objects and environments via 3D scanners, generative models, and 3D reconstruction.
  - **5.1.1** Scan operation objects and robot environments with 3D scanners.
  - **5.1.2** Generate 3D models from internet images via SAM3D, and generate different surface textures using diffusion models.
  - **5.1.3** Reconstruct environment 3D models from internet videos via VGGT/ DepthAnythingv3.
  - **5.1.4** Convert models from steps 2–3 to point clouds and manually denoise.
- **5.2** Build simulation environments and load the above 3D models with robot URDFs.
- **5.3** Randomize object positions, textures, angles, and lighting in simulation.
- **5.4** Set camera intrinsics/extrinsics, initial robot pose, end-effector pose, and initial hand joint angles.

5.5 Predict robot trajectories via the action model based on primitive action sequences for various tasks.

5.6 Dynamically adjust motion trajectories via trajectory planning algorithms to reach near objects.

5.7 Dynamically adjust end-effector poses via inverse kinematics to reach the target near objects.

5.8 Dynamically adjust hand joint angles and compute SDF values between the hand and object 3D models to minimize SDF while avoiding interpenetration.

$$\min_{\mathbf{q}_h} \ \mathrm{SDF}\big(\mathrm{hand}(\mathbf{q}_h), \mathrm{object}\big), \qquad \text{s.t. } \mathrm{SDF}\big(\mathrm{hand}(\mathbf{q}_h), \mathrm{object}\big) \geq \epsilon_{\mathrm{contact}}, \quad (45)$$

where the constraint prevents interpenetration.

5.9 For mobility primitive action, judge success based on whether the object is within the robot's reachable workspace.

$$\mathrm{success}_{\mathrm{mobility}} = \mathbb{1}\big[\mathbf{o}_{\mathrm{obj}} \in \mathcal{W}_{\mathrm{reachable}}(\mathbf{q}_{\mathrm{robot}})\big]. \quad (46)$$

5.10 For manipulation primitive action, judge success by whether the SDF between hand and object is below a threshold with no interpenetration.

$$\mathrm{success}_{\mathrm{manip}} = \mathbb{1}\Big[\mathrm{SDF}\big(\mathrm{hand}, \mathrm{object}\big) < \tau_{\mathrm{sdf}} \ \wedge \ \text{no interpenetration}\Big]. \quad (47)$$

5.11 Supervise the action model's trajectory outputs, success/failure probabilities, and future world features via reinforcement learning [13].

$$\max_{\theta} \ J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right], \quad (48)$$

where the action model π_θ provides trajectory, value, and feature predictions that receive reinforcement supervision.

5.12 Randomly load different robot URDFs, different objects, random obstacles, and randomize initial poses, camera views, object positions, lighting, and textures; use Rapidly-exploring Random Trees to generate large numbers of diverse motion and manipulation trajectories, repeating steps 3–11.

5.13 Record simulation data (multi-view rendered video, primitive action type, robot pose, end-effector/hand joint trajectories, tactile info, success flags, etc.) in real time and save in H5 format.

3.3 Algorithm Architecture

3.3.1 Perception

3.3.1.1 Visual Perception Through image/video/RGB-D/lidar point cloud inputs, perceive object boundaries, types, keypoints, recognition features, and 3D spatial positions, addressing core issues of proprioceptive and environmental perception (where and what) [11][12].

$$\mathbf{o} = \Big(\underbrace{\mathbf{p}_{3\mathrm{D}}}_{\text{where}}, \ \underbrace{c}_{\text{what}}, \ \mathbf{b}, \ \{\mathbf{k}_j\}, \ \mathbf{f}_{\mathrm{rec}}\Big), \quad (49)$$

where $\mathbf{p}_{3D}$ is the 3D position, c the object type, $\mathbf{b}$ the boundary, $\{\mathbf{k}_j\}$ keypoints, and $\mathbf{f}_{\text{rec}}$ the recognition feature.

Visual hierarchical models:

- Detection: bounding boxes (bbox/obb, anchor-based/anchor-free); engineering: Ultralytics.
- Segmentation: semantic, instance, panoptic, and salient object segmentation; models: SAM, UNet, DeepLabV3+.
- Classification: object, scene, image quality, object quality, video classification; engineering: ClassyVision.
- Keypoints: coordinate regression or heatmap regression for faces, bodies, hands, objects; engineering: Ultralytics.
- Recognition: feature extraction to cluster same classes and separate different classes, then compare with a registration set; includes face recognition, person re-ID, generic object recognition, speech recognition, OCR; engineering: InsightFace.

$$\hat{c} = \arg\max_{c \in \mathcal{C}_{\text{reg}}} \ \text{sim}\big(\mathbf{f}(x), \mathbf{f}_c^{\text{reg}}\big), \tag{50}$$

  where $\mathcal{C}_{\text{reg}}$ is the registration set and $\mathbf{f}_c^{\text{reg}}$ denotes registered features of class c.

- Pose estimation: regression (e.g., edgeai-yolox) or geometry (points + ICP) + model-based (e.g., FoundationPose, ZebraPose).
- Depth estimation: monocular (GlpDepth, ZipDepth), completion (lingbot-depth), stereo (geometric calibration + rectification + disparity + triangulation; model-based: FoundationStereo, RT-Monster), video/multi-view (VGGT, DA3, lingbot-map).

$$z = \frac{bf}{d}, \tag{51}$$

  where z is depth, b the stereo baseline, f the focal length, and d the disparity.

- Point cloud recognition: PointNet++, PointPillars, CenterPoint.
- 3D reconstruction: SFM, MVS, NeRF, 3DGS.
- Surround-view (BEV, OCC) is more relevant for autonomous vehicles and not essential for most robots.

Visual multi-task models (e.g., DINO, HydraNet) simultaneously output detection, segmentation, classification, keypoints, depth, etc.

$$\big(\mathbf{b}, c, \mathbf{k}, \mathbf{f}, z\big) = \text{MultiTask}(I; \theta), \tag{52}$$

where I is an input image and θ denotes the shared multi-task model parameters.

SLAM:

- Odometry-based: visual, LiDAR, IMU, current pose = initial pose + cumulative odometry; algorithms: SuperPoints+LightGlue, LIO.

$$\mathbf{p}_t = \mathbf{p}_0 + \sum_{k=1}^{t} \Delta\mathbf{p}_k, \tag{53}$$

where $\Delta\mathbf{p}_k$ is the incremental odometry at step k.

- Landmark-based: marker, key object recognition and pose estimation, current pose = landmark pose − relative pose between robot and landmark; algorithms: ORB-SLAM3 etc.

$$\mathbf{p}_{\text{robot}} = \mathbf{p}_{\text{landmark}} - \Delta\mathbf{p}_{\text{robot from landmark}}, \tag{54}$$

where $\Delta\mathbf{p}_{\text{robot from landmark}}$ is the relative pose of the robot with respect to the landmark.

- Obstacle avoidance: obstacle extraction from point clouds (LiDAR/RGB-D/depth), detection/segmentation for known sets, drivable area segmentation for open-set obstacles.

- Localization bias: GPS (meters), scene-map localization (cm), relative boundary segmentation (mm), tactile sensors (dmm).

- Classic SLAM algorithms:
  - Cartographer: Indoor service robots / warehouse AGVs, Industrial-grade stability, excellent large-scale mapping consistency;
  - Gmapping: Small-scale indoor robots, Low computation, high accuracy, well integrated with ROS;
  - FAST-LIO2 or Hector SLAM: UAVs / handheld devices, No odometry needed, high computational efficiency, adapts to high-speed motion;
  - LOAM / LIO-SAM: High-precision 3D mapping, Benchmark for 3D LiDAR SLAM, high accuracy;
  - ORB-SLAM3: Vision-dominant scenes, Best accuracy among visual SLAM, supports multi-sensor configurations;
  - VINS-Mono / fusion: Visual-IMU fusion, Tightly coupled VIO with excellent performance;
  - R3LIVE: Colored 3D mapping / surveying, Real-time colored point clouds, dense reconstruction;
  - Karto SLAM: Resource-constrained platforms, High CPU efficiency;
  - Multi-sensor fusion: sensors: vision, LiDAR, IMU, RTK, GPS; algorithms—ORB-SLAM3 + LiDAR (e.g., visual-inertial-slam-fusion).

3.3.1.2 Tactile Perception Use tactile sensor information to determine whether the robot/hand has contacted the object and the contact force magnitude; reference projects: ViTacFormer, 3DTacDex, PP-Tac.

$$\big(c_{\text{contact}},\ F_{\text{contact}}\big) = \text{TactileModel}\big(\mathbf{s}_{\text{tac}}\big), \tag{55}$$

where $c_{\text{contact}} \in \{0, 1\}$ indicates contact and F_{contact} is the contact force magnitude.

3.3.1.3 Other Modalities: auditory, gustatory, olfactory, etc.

3.3.2 Planning and Control

3.3.2.1 Planning Classical planning algorithms include global planning (A*, Dijkstra, RRT, genetic algorithms) and local planning (DWA, TEB, PID tracking, artificial potential fields).

$$\tau^* = \arg\min_{\tau \in \mathcal{T}} \; J_{\text{cost}}(\tau), \tag{56}$$

where τ denotes a trajectory and J_{cost} is the planning cost. Robotics motion planning libraries such as Pinocchio, PyRoboPlan, Pyroki, and MoveIt provide standardized implementations for manipulator motion planning. Model-based planning: MPC, VN, VLN provide trajectory planning in dynamic environments.

$$\tau^*_{t:t+H} = \arg\min_{\tau_{t:t+H}} \; J\big(\mathbf{s}_t, \tau_{t:t+H}\big), \tag{57}$$

where $\mathbf{s}_t$ is the current state and the plan is re-evaluated in a receding-horizon manner in dynamic environments. LLM/VLM planning represents a new paradigm—through language or visual descriptions, large models output structured planning language or JSON-format sequences of primitive actions [17].

$$a^{\text{plan}}_{1:T} = \text{LLM/VLM}(l, v), \qquad a_t \in \mathcal{A}_{\text{primitive}}, \tag{58}$$

where l is a language description, v is a visual description, and $\mathcal{A}_{\text{primitive}}$ is the primitive-action set.

3.3.2.2 Control Mobility control: wheeled chassis, rotorcraft, quadruped, wheeled-quadruped, biped, wheeled-biped, etc. Manipulator + dexterous hand/gripper operation:

- Hierarchical grasping: what to grasp, where to grasp, and how to grasp (object detection/recognition, pose estimation, grasp gesture and trajectory planning) [14].
- Object instance segmentation: YOLO/SAM.
- Stereo depth estimation: RT-Monster/FoundationStereo.
- Solve object pose in camera frame and transform to robot base frame: $T^{\text{base}}_{\text{end}} = T^{\text{base}}_{\text{head}}\, T^{\text{head}}_{\text{camera}}\, T^{\text{camera}}_{\text{end}}$, where $T^{\text{camera}}_{\text{end}}$ comes from segmentation+depth model, $T^{\text{head}}_{\text{camera}}$ from calibration, $T^{\text{base}}_{\text{head}}$ from forward kinematics via URDF.

  $$T^{\text{base}}_{\text{end}} = T^{\text{base}}_{\text{head}} \cdot T^{\text{head}}_{\text{camera}} \cdot T^{\text{camera}}_{\text{end}}, \tag{59}$$

  where the superscript and subscript denote the source and target frames, respectively.
- Grasp computation: compute SDF values from object geometry + hand/gripper URDF, ICP between fingers/palm and object point cloud; model-based: DexGraspNet.

  $$\mathbf{q}^*_h = \arg\min_{\mathbf{q}_h} \Big[\text{SDF}\big(\text{hand}(\mathbf{q}_h), \text{object}\big) + \lambda\, \text{penetration}\big(\text{hand}(\mathbf{q}_h), \text{object}\big)\Big], \tag{60}$$

  where the first term favors surface contact and the second penalizes interpenetration.

- Solve relative pose from key fingertips (thumb/index) to object center.
- Solve relative pose from fingertips to end-effector.
- Plan trajectory from initial end-pose to target end-pose.
- Robot-specific variation problem: cumulative errors from camera calibration, manufacturing/assembly tolerances, zero-position calibration errors, actuation errors, perception localization errors.

$$\epsilon_{\text{total}} = \epsilon_{\text{calibration}} + \epsilon_{\text{assembly}} + \epsilon_{\text{zero}} + \epsilon_{\text{actuation}} + \epsilon_{\text{perception}}, \tag{61}$$

  where the components denote the corresponding cumulative error sources.
- Solutions: environmental perception + proprioception + visual relative offset control + tactile motion compensation + PID control.

Reinforcement Learning: MCTS, DQN, PPO, GRPO, RLHF, etc. [13].

$$\theta \leftarrow \theta + \alpha \, \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \gamma^t r_t \right], \tag{62}$$

where the expectation is over trajectories τ and the concrete estimator depends on the chosen algorithm (e.g., MCTS, DQN, PPO, GRPO, RLHF).

3.3.3 End-to-End Models

End-to-end models (VA, VLA, VTA, VTLA, world models, e.g., LeRobot, Cosmos) have gained much attention [11][12]. However, global end-to-end models face challenges: excessive complexity, cold-start data issues, multimodal coupling, missing modalities, poor interpretability, and difficult regression testing for new versions. We advocate a "first layer-wise, then end-to-end" progressive strategy: first establish layered perception-planning-control systems to solve known problems; as execution data accumulates, gradually expand task types and scenarios and convert high-frequency tasks to end-to-end—trading data for efficiency, reducing information loss, and increasing theoretical upper bounds.

$$\mathcal{M}_{k+1} = \begin{cases} \mathcal{M}_{\text{layered}} = \mathcal{C} \circ \mathcal{P} \circ \mathcal{E}, & \text{if execution data } d_k < d^*, \\ \text{E2E}\big(\mathcal{M}_{\text{layered}}, \mathcal{D}_{1:k}\big), & \text{if high-frequency tasks accumulate, } d_k \geq d^*, \end{cases} \tag{63}$$

where $\mathcal{D}_{1:k}$ is the accumulated execution data and d^* is the data threshold. This aligns with how humans learn complex tasks: initially breaking down steps, and with repeated execution and exposure to anomalies, eventually performing them fluidly.

3.3.4 Language and Generation

Language modules cover: listening (ASR), speaking (TTS), reading (OCR), and writing (LLM/VLM + RAG). Retrieval-augmented generation (RAG) significantly boosts domain-specific performance through the synergy of Embedding, Rerank, Search, and Prompt.

$$\mathbf{y} = \text{LLM}\Big(\text{Prompt} \ \oplus \ \text{Rerank}\big(\text{Retrieve}_{\text{embed}}(q)\big)\Big), \tag{64}$$

where q is the query and $\oplus$ denotes concatenation of the prompt and retrieved context. Generative models include GANs, diffusion, flow matching, drifting, etc., playing increasingly important roles in image, video, audio, and action generation.

3.3.5 Multi-Level MoE Architecture

Mixture-of-Experts (MoE) provides an elegant scaling method for general-purpose agents: information is routed through a Router, activating multiple Encoders; fused features are then output by multiple Decoders. Encoders are divided into shallow, medium, and deep layers; Decoders support coarse-, medium-, and fine-grained outputs. The Router dynamically selects which Encoders/Decoders to activate, which layers of each Encoder to use, and which granularity of each Decoder to output, based on input type, complexity, and required accuracy.

$$\mathbf{f}_{\text{fused}} = \sum_i g_i^E(\mathbf{x})\,\text{Encoder}_i^{\ell_i}(\mathbf{x}), \qquad \mathbf{y} = \sum_j g_j^D(\mathbf{f}_{\text{fused}})\,\text{Decoder}_j^{r_j}(\mathbf{f}_{\text{fused}}), \tag{65}$$

where $g_i^E(\mathbf{x})$ and $g_j^D(\mathbf{f}_{\text{fused}})$ are router weights depending on input type, complexity, and required accuracy; ℓ_i is the selected encoder layer depth and r_j the selected decoder output granularity.

3.3.6 Agents and Continual Learning

The core of an agent system includes knowledge base construction, dynamic model planning, execution feedback, and continual learning [3][4][5]. The knowledge base comprises: atomic skill libraries (modular encapsulation of functions/classes), skill trees, databases (vector, relational, graph), workflows, execution dependencies, issue repositories, and version management of all modules.

$$\mathcal{K} = \big\{\mathcal{A}_{\text{skills}},\ \mathcal{T}_{\text{skill tree}},\ \mathcal{D}_{\text{vec/rel/graph}},\ \mathcal{W}_{\text{workflows}},\ \mathcal{D}_{\text{exec}},\ \mathcal{I}_{\text{issues}},\ \mathcal{V}_{\text{versions}}\big\}. \tag{66}$$

Executable program generation adopts a "problem + RAG + LLM" pattern, combined with dynamic execution, anomaly detection, behavior correction, and feedback mechanisms, forming a closed-loop for continuous evolution.

$$\begin{aligned} &\text{program} = \text{LLM}\big(\text{problem} \ \oplus\ \text{RAG}\big), \\ &\quad \text{loop}: \ \text{execute} \to \text{anomaly detect} \to \text{correct} \to \text{learn}. \end{aligned} \tag{67}$$

Continual learning is the key capability that distinguishes general-purpose agents from static models. Key techniques include: test-time tuning, Elastic Weight Consolidation (EWC), experience replay, sub-network training, and integrated training-inference architectures.

$$\mathcal{L}_{\text{CL}}(\theta) = \mathcal{L}_{\text{new}}(\theta) + \frac{\lambda}{2}\sum_i F_i\big(\theta_i - \theta_i^*\big)^2, \tag{68}$$

where F_i is the Fisher information of old tasks at parameter θ_i^* and λ controls the consolidation strength.

3.4 Compute and Deployment Engineering

The realization of intelligence relies on compute support. From data processing to model training (PyTorch, TensorFlow), from model lightweighting (neural architecture search, distillation, pruning, QAT, PTQ) to platform adaptation (CPU, GPU, NPU),

and from inference frameworks (TensorRT, FlashRT, TNN, MNN, NCNN, RKNN) to SDK development—this constitutes the complete compute pipeline.

$$\begin{aligned}\text{Compute pipeline}: \quad &\text{data} \to \text{train} \to \text{lightweight} \\ &\to \text{platform adaptation} \to \text{infer} \to \text{SDK}.\end{aligned} \tag{69}$$

Edge-cloud integration is the current mainstream deployment mode: on-device deployment of small models (using Ollama, vLLM, TensorRT-LLM, FlashRT, RKLLM, etc.) ensures low latency and data security, while cloud APIs (GPT, Claude, DeepSeek, etc.) provide on-demand access to very large models.

$$r \mapsto \begin{cases}\text{edge}(r), & \text{small model, low latency, private data,} \\ \text{cloud}(r), & \text{large model accessed on demand,}\end{cases} \tag{70}$$

where r denotes a request. Deployment SDKs must include a general scheduling architecture, communication (ROS2/Socket), and safety mechanisms (hardware constraints, logic constraints, exception handling).

General brain + specific actuators architecture: Brain: perception, planning, control, end-to-end, language, generation, MoE, agent, continual learning. Workflow: data processing, training, inference, deployment, runtime, feedback data, and software-hardware iteration closed-loop. Actuators: mobility (wheeled, rotorcraft, quadruped, wheeled-quadruped, biped, wheeled-biped, etc.), body (folding joints, lifting rods, etc.), manipulation (hands, grippers, forks, turntables, supports, etc.).

$$\begin{aligned}\text{System} &= \langle \text{Brain},\ \text{Workflow},\ \text{Actuators} \rangle, \\ \text{Workflow}: &\ \text{data} \to \text{train} \to \text{deploy} \to \text{runtime} \to \text{feedback} \to \text{iterate}.\end{aligned} \tag{71}$$

3.5 Current Status and Shortcomings

Although all key modules for general intelligence are available, significant gaps remain toward truly general intelligence [16]. First, insufficient integration—each basic module has multiple independent implementations, but simple concatenation introduces large redundancy. There is a lack of a universal feature encoder (taking different modalities and outputting features that cluster within classes and separate across classes) and a universal decoder (decoding fused features to different downstream tasks with variable granularities).

$$\nexists\, \mathcal{E}^*: \ \mathcal{M} \to \mathcal{F} \quad \text{s.t.} \quad \text{sim}\big(\mathcal{E}^*(x_i), \mathcal{E}^*(x_j)\big) \ \begin{cases}\text{large}, & y_i = y_j, \\ \text{small}, & y_i \neq y_j,\end{cases} \tag{72}$$

and similarly there is no universal decoder $\mathcal{D}^*$ that maps fused features to arbitrary downstream tasks and granularities. Second, low data efficiency—data generation and real-robot iteration are inefficient; a unified multimodal data generation pipeline is missing. Language-modality data is relatively rich, but effective data for visual-spatial, tactile, and action modalities are still insufficient. Third, hardware bottlenecks—hardware stability, especially of dexterous hands, has significant room for improvement. Fourth, inadequate continual learning—the integration of long-term and short-term goal setting, value networks, and autonomous exploration requires deeper investigation.

$$\text{CL}^* \ \supseteq \ \big\{\text{long/short-term goal setting},\ \text{value networks},\ \text{autonomous exploration}\big\}. \tag{73}$$

4 Evaluation of Intelligence

4.1 Evaluation Dimensions

Intelligence can be assessed along five dimensions [1][2]:

$$\mathbf{Q} = \big(P, S, D, B, E\big), \qquad P, S, D, B, E \in \mathbb{R}_+, \tag{74}$$

where P is precision, S speed, D depth, B breadth, and E efficiency.

1. **Precision**: accuracy and reliability of task completion, corresponding to the accuracy of neural network outputs.

$$P = \frac{\text{number of correctly completed tasks}}{\text{total number of tasks}}. \tag{75}$$

2. **Speed**: timeliness of information processing and decision-making, corresponding to training and inference speed on specific hardware.

$$S = \frac{1}{t_{\text{inference}}} \tag{76}$$

where $t_{\text{inference}}$ is the per-decision inference time (equivalently, throughput on a given hardware platform).

3. **Depth**: depth of understanding of complex problems, corresponding to network depth, which determines extraction and processing of higher-order features.

$$D \propto L_{\text{network}}, \qquad L_{\text{network}} = \text{number of processing layers}, \tag{77}$$

where deeper networks are expected to extract and process higher-order features.

4. **Breadth**: cross-domain and cross-modal generalization, corresponding to network width, enabling parallel processing and multimodal fusion.

$$B \propto W_{\text{network}} \times M_{\text{tasks/modalities}}, \tag{78}$$

where W_{network} is network width and $M_{\text{tasks/modalities}}$ is the number of tasks or modalities handled.

5. **Efficiency**: effective output per unit of resource (matter, energy, information, time, space), corresponding to effective output with limited compute, power, data, inference time, and robot workspace.

$$E = \frac{\text{effective output (task success)}}{\text{resource consumption (compute, power, data, time, space)}}. \tag{79}$$

Basic intelligences are somewhat independent; excellence in one does not guarantee excellence in another—for example, Newton was pre-eminent in physics but not in stock trading. Even within the same domain, different forms of intellect can be independent—e.g., the mathematician Hermite made major research contributions yet performed poorly in mathematical examinations. This resembles an intelligent machine with ordinary CPU, memory, storage, and communication, but equipped with powerful GPU/NPU and strong

perception/generation models, given external valid information, it shows high perception, imagination, and creativity, but poor offline memory, search, and computation. Contemporary examinations, however, mainly test offline memory, search, and computation.

$$q_i \not\Rightarrow q_j \qquad (i \neq j), \quad \mathbf{q} = (P, S, D, B, E), \tag{80}$$

meaning high performance along one dimension or form of intelligence does not guarantee high performance along another.

4.2 Comparison Between Human and Machine Intelligence

In terms of memory, search, computation efficiency, and knowledge dissemination, AI has far surpassed humans. Large language models can retrieve and process text volumes in seconds that a human could not read in a lifetime [18]. In terms of imagination efficiency, text-, image-, and speech-based generation are approaching human levels, but multimodal imagination and generation still lag. In complex environment perception/recognition, creativity, energy efficiency, and real-world adaptability, AI still has a significant gap from humans [1], the human brain operates at 20 watts for remarkable cognitive tasks, while training a large model requires megawatt-scale compute.

$$P_{\text{human brain}} \approx 20 \text{ W}, \qquad P_{\text{large model training}} \sim 10^6 \text{ W}, \tag{81}$$

$$\eta_{\text{cognition}} = \frac{\text{effective cognitive output}}{P}, \qquad \eta_{\text{human}} \gg \eta_{\text{machine}} \text{ at present.} \tag{82}$$

However, this gap can be narrowed. By building a general-purpose brain with full perception, planning, control, autonomous exploration, and continual learning, combined with diverse effectors and rapid iteration in numerous real-world scenarios, AI's intelligence deficiencies can be progressively eliminated. The theoretical upper bound of machine environmental adaptability is far higher than humans, they can survive in environments lethal to humans, evolve at speeds impossible for humans, and scale indefinitely [19].

$$\sup_{E \in \mathcal{E}} \mathcal{A}_{\text{machine}}(E) \;\geq\; \sup_{E \in \mathcal{E}} \mathcal{A}_{\text{human}}(E), \tag{83}$$

where $\mathcal{A}(\cdot)$ is environmental adaptability and $\mathcal{E}$ denotes the set of possible environments; the machine can also evolve and scale without human biological constraints.

5 Conclusion and Outlook

Starting from the core thesis that "intellect is mapping," this paper systematically constructs a theoretical framework and implementation pathways for intelligence. We have argued for the essence of "intellect as information filtering, compression, storage, association, transformation, and generation, and for the synergy of intellect, knowledge, and resources in forming capabilities.

$$\mathcal{I} : \mathcal{X} \to \mathcal{Y}, \qquad C = \mathcal{F}\big(\mathcal{I}, K, R\big), \qquad \mathbf{Q} = (P, S, D, B, E). \tag{84}$$

On the implementation side, we elaborated on the modular architecture of general-purpose agents—from elemental faculties (attention, perception, memory, search, computation, imagination) to combined intelligences (recognition and creativity), and then to the hierarchical progression of embodied, collective, and machine intelligence.

Our core viewpoints can be summarized in three points: First, the key modules for general intelligence are already available; the real challenge lies in system-level integration —requiring unified feature representations, general encoding-decoding architectures, and efficient autonomous exploration and continual learning mechanisms. Second, data is the primary bottleneck—whether sufficiently rich data can be obtained, whether valuable mapping relationships exist within the data, and the efficiency of data iteration determine the ultimate performance of intelligent systems. Third, the progressive "first layer-wise, then end-to-end" strategy is a pragmatic path toward general intelligence—accumulating data and experience in a layered architecture, then gradually converting mature tasks to end-to-end.

Looking ahead, the following directions merit special attention:

1. General feature representation learning—building a unified cross-modal, cross-task feature space.

2. Efficient data generation and sim-to-real transfer—closing the gap between simulation and reality.

3. Continual learning and autonomous exploration—enabling agents to autonomously set goals and continuously evolve in open environments.

4. Neuro-symbolic fusion architectures—incorporating the structuredness and interpretability of symbolic reasoning while retaining neural flexibility [8][15].

5. Software-hardware co-optimization—vertical integration from algorithms to chips for improved energy efficiency.

General artificial intelligence is on the horizon, and the direction is already clear. Intelligence is the key for life to unlock higher energy levels, and the ticket for human civilization to reach the stars. In the evolution of the universe, stable and efficient entropy-increasing structures have a higher probability of being preserved, the trend is independent of human will, but how to align with it while managing risks remains a question.

References

[1] Levin's continuum perspective proposes an alternative: Machine Psychometrics: A Mathematical Psychology of Artificial Intelligence. ar5iv.labs.arxiv.org.

[2] Matta, D. FROM THRESHOLDS TO TRAJECTORIES: The AGI Capability Maturity ModelTM for Artificial General Intelligence. PhilArchive, 2025.

[3] A Comprehensive Survey on Agentic Artificial Intelligence Architectures: Reasoning, Planning and Real-World Applications. International Journal of Advanced Multidisciplinary Application, 2025.

[4] Xu, B. AI Agent Systems: Architectures, Applications, and Evaluation. JACM, 2025.

[5] A Holistic Review of Agentic AI Frameworks, Applications, and Research Trajectories. Archives of Computational Methods in Engineering, 2026.

[6] Zhang, W., et al. Embodied AI: A Survey on the Evolution from Perceptive to Behavioral Intelligence. SmartBot, 2025.

[7] Xu, Z., et al. A Survey on Robotics with Foundation Models: toward Embodied AI. arXiv:2402.02385, 2024.

[8] Nawaz, U., et al. A review of neuro-symbolic AI integrating reasoning and learning for advanced cognitive systems. Intelligent Systems with Applications, 2025.

[9] A Comparative review of Symbolism and Connectionism in AI&Law: Tracing evolution and exploring integration. ScienceDirect, 2025.

[10] Mokaria, N. A Survey on Foundations and Frontiers of Multimodal Agentic Frameworks: Techniques and Applications. arXiv:2608.20379, 2026.

[11] Exploring Embodied Multimodal Large Models: Development, datasets, and future directions. Information Fusion, 2025.

[12] Foundation Models in Robotics: A Comprehensive Review of Methods, Models, Datasets, Challenges and Future Research Directions. arXiv, 2026.

[13] A Comprehensive Survey of Advances in Deep Reinforcement Learning for Autonomous Robotics Applications. Zenodo, 2025.

[14] Embodied Robot Manipulation in the Era of Foundation Models: Planning and Learning Perspectives. arXiv, 2026.

[15] Neuro-symbolic agentic AI: Architectures, integration patterns, applications, open challenges and future research directions. ScienceDirect, 2026.

[16] The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning. arXiv, 2026.

[17] Large language models for robotics: Opportunities, challenges, and perspectives. ScienceDirect, 2024.

[18] The Standard Model of Mind. In: Generative AI vs. AGI: The Cognitive Strengths and Weaknesses of Modern LLMs. ar5iv.labs.arxiv.org.

[19] Open General Intelligence (OGI) Framework. arXiv.